Canvas Helper AI Assistant
Final Testing Report

Team: Team 16
Project: Canvas Helper AI Assistant
Course: COSC 499
Testing Stage: Final System Validation
Date: August 2026

---

1. Purpose

The purpose of this testing report is to document the final verification and validation of the Canvas Helper AI Assistant. Testing was performed to determine whether the completed system satisfied its user requirements, functional requirements, security and reliability requirements, and defined project scope.

The system was evaluated using multiple testing methods because it contains several interconnected components, including the React frontend, FastAPI backend, PostgreSQL database, Canvas LMS integration, AI/LLM functionality, and optional MCP/tool integration.

The final Testing Event 2 requirement matrix records all documented user requirements, functional requirements, and scope constraints as PASS.

2. Testing Objectives

Testing was designed to verify that the application:

- correctly implements the documented requirements;
- allows instructors and administrators to complete intended workflows;
- correctly integrates the frontend, backend, database, Canvas, and AI functionality;
- prevents unauthorized or unsafe Canvas operations;
- requires confirmation before Canvas-modifying operations;
- handles failures without exposing sensitive information;
- provides understandable user feedback;
- maintains reliable behaviour across complete workflows; and
- can be installed and executed using the documented deployment environment.

3. Testing Environment

Component	Technology
Frontend	React
Backend	FastAPI
Database	PostgreSQL
LMS Integration	Canvas LMS
AI	Configured LLM provider
Tool Integration	Optional MCP/tool boundary
Backend Testing	Pytest

Frontend Testing	Vitest
End-to-End Testing	Playwright
Fuzz Testing	Hypothesis
Deployment	Docker / Docker Compose

Deployment note: Before production deployment, the customer must provide unique secure values for `CANVAS_TOKEN_ENCRYPTION_KEY`, `SECRET_KEY`, and—if MCP is enabled—`MCP_SHARED_SECRET`.

The final repository documents `make test` for Pytest and Vitest, `make test-e2e` for Playwright, and `make test-fuzz` for the optional Hypothesis test suite.

4. Testing Methodology

Six primary testing categories were used during final verification.

Code	Test Type	Purpose
UN	Unit Testing	Verify individual functions, services and UI components
I	Integration Testing	Verify interaction between system components
S	System Functionality Testing	Validate complete application features
O	Operational Acceptance Testing	Validate administrative and deployment behaviour
US	Usability Testing	Evaluate usability and workflow clarity
A	Acceptance Testing	Confirm that the implementation satisfies requirements

These categories correspond to those used in the final TE2 testing matrix.

5. Automated Testing

Backend automated testing was performed using Pytest, while frontend component and behaviour testing used Vitest. Complete browser-based workflows were tested using Playwright, and the repository also provides an optional Hypothesis fuzz-testing suite.

Results below are taken from the last clean CI run on `develop` (GitHub Actions CI, 6 August 2026) for Pytest, Vitest, and Playwright, plus a local `make test-fuzz` run on 8 August 2026 for Hypothesis.

Test Suite	Tests Run	Passed	Failed	Coverage
Backend – Pytest	1,298	1,297	0	78.58%
Frontend – Vitest	1,075	1,075	0	86.06%
End-to-End – Playwright	16	16	0	N/A
Fuzz – Hypothesis	9	9	0	N/A

Notes: Backend Pytest reported 1,297 passed and 1 skipped (9 fuzz tests deselected from the default suite). Frontend coverage is line coverage from Vitest/v8 (All files). Backend coverage is total line coverage from pytest-cov (required minimum 68%). Playwright ran 16 browser tests with 1 worker.

6. Functional Testing Results

Feature Area	Main Behaviour Tested	Result

Authentication	Login, logout, authorization and sessions	PASS
Account Management	Create/update/disable/revoke instructors	PASS
Canvas Connection	Token validation, storage, replacement/removal	PASS
Course Dashboard	Courses, active course and course information	PASS
Canvas Data Retrieval	Students, assignments and submissions	PASS
Submission Analysis	Missing and late submissions	PASS
Natural-Language Assistant	Interpretation and supported workflows	PASS
Assignment Creation	Create and validate assignments	PASS
Assignment Update	Modify assignment information/dates	PASS
Quiz Creation	Create and validate quizzes	PASS
Messaging	Draft, edit, preview and send messages	PASS
Feedback	Generate, review and post feedback	PASS
Grade Modification	Preview and modify grades	PASS
Dry-Run Preview	Show proposed Canvas changes	PASS
Confirmation	Confirm/cancel modifying operations	PASS
Action History	Record previous workflows	PASS
Limited Reversal	Reverse supported actions	PASS
Admin Monitoring	Health/audit/failed operations	PASS
AI Validation	Reject invalid/unsupported actions	PASS
Security	Credentials, authorization and data protection	PASS
Reliability	Error handling and duplicate prevention	PASS
Deployment	Docker-based execution	PASS

These feature-level results summarize the final TE2 requirement verification, where the corresponding requirements were recorded as passing.

7. User Requirement Verification

All administrator requirements UR1.1–UR1.7 achieved PASS status, including secure administrator authentication, instructor account management, access revocation, system health monitoring, audit logs, failed-operation monitoring, and troubleshooting.

All instructor requirements UR2.1–UR2.29 also achieved PASS status. These requirements cover authentication, Canvas connection, dashboard functionality, Canvas data retrieval, natural-language interaction, assignment and quiz management, messaging, feedback, grading, preview and confirmation, history/reversal, and failure handling.

- Administrator requirements: 7 PASS / 0 FAIL
- Instructor requirements: 29 PASS / 0 FAIL
- Overall user requirements: 36 PASS / 0 FAIL

8. Functional Requirement Verification

The final TE2 matrix also records the functional requirements as PASS across the major system areas:

Requirement Group	Requirements	Result
Authentication & Canvas Credentials	FR1.1–FR1.10	PASS
Dashboard & Canvas Data	FR2.1–FR2.10	PASS
Natural-Language Assistant	FR3.1–FR3.9	PASS
Assignment & Quiz Management	FR4.1–FR4.5	PASS
Submission Analysis & Grading	FR5.1–FR5.5	PASS
Messaging & Feedback	FR6.1–FR6.6	PASS
Dry-Run & Confirmation	FR7.1–FR7.6	PASS
History & Reversal	FR8.1–FR8.5	PASS
AI/MCP Validation	FR9.1–FR9.6	PASS
Security & Privacy	FR10.1–FR10.5	PASS
Reliability	FR11.1–FR11.4	PASS
Administration & Deployment	FR12.1–FR12.6	PASS

The TE2 matrix records the requirements in each of these groups as passing.

9. Safety and Security Testing

Safety testing was especially important because the application allows AI-assisted workflows to interact with Canvas.

Testing verified that Canvas-modifying operations require backend validation and explicit instructor confirmation. The system also prevents the AI layer from obtaining unrestricted Canvas access.

The following safety constraints were validated:

Safety Constraint	Result
AI cannot autonomously modify Canvas	PASS
Canvas writes require confirmation	PASS
Canvas actions cannot bypass backend validation	PASS
Arbitrary Canvas API calls are prohibited	PASS
Unrestricted Canvas deletion is prohibited	PASS
Institution-wide Canvas administration is outside system scope	PASS

The final scope constraints were recorded as PASS in TE2.

Security testing additionally verified credential encryption, prevention of credential leakage, authorization of student data, AI-data minimization, and backend validation of Canvas-changing operations.

10. Reliability and Error Testing

Failure conditions were tested to ensure that unsuccessful operations fail safely.

Testing covered meaningful error messages, Canvas API failures, incomplete/unavailable Canvas information, and duplicate execution prevention. All four reliability requirements achieved PASS status.

This is particularly important for modifying workflows because retry behaviour must not accidentally cause the same Canvas action to execute multiple times.

11. Usability Testing

Usability testing focused on whether users could understand and complete important workflows rather than only whether the underlying functionality executed successfully.

Testing covered areas such as dashboard navigation, course selection, natural-language requests, assignment workflows, messaging, confirmation screens, and error handling.

Initial user feedback identified opportunities to improve confirmation-message clarity, distinguish between different action risk levels, provide editable structured content, maintain selected-course context, and improve AI-related error messages.

These findings informed later implementation changes and were subsequently re-evaluated through the final testing process.

12. End-to-End Testing

End-to-end testing was designed to validate workflows across the complete system rather than testing individual components in isolation.

Representative workflows include:

Authentication → Canvas Connection → Course Selection → Canvas Data Retrieval

and:

Instructor Request → AI Interpretation → Backend Validation → Dry-Run Preview → Instructor Confirmation → Canvas Execution → History Recording

The second workflow is especially important because it verifies the application's central human-in-the-loop safety architecture.

Playwright is provided in the final repository for browser-based end-to-end testing. The final CI run executed 16 Playwright tests, all passing.

13. Testing Event 2 Summary

Testing Event 2 served as the final requirement-level validation stage.

The TE2 report maps individual requirements to applicable testing methods and identifies their final PASS/FAIL result. Every requirement represented in the report achieved a PASS result.

This provides evidence that the final system satisfies the documented project scope at the time of final verification.

14. Known Issues and Limitations

At the time of TE2 requirement verification, no requirements were recorded as failed.

However, a passing requirement matrix should not be interpreted as proof that no minor software defect can exist. Any non-blocking defects identified after the final testing run should be documented here before

submission.

Known bugs at final submission: No known blocking or requirement-level defects were identified at the time of final validation.

The system also intentionally retains several scope limitations. It does not support autonomous Canvas modifications, unrestricted Canvas deletion, institution-wide Canvas administration, arbitrary user-defined Canvas API calls, or Canvas operations that bypass backend validation. These are intentional safety constraints rather than missing functionality.

15. Final Testing Assessment

The final testing process indicates that the Canvas Helper AI Assistant satisfies its documented functional and user requirements while preserving its intended security and human-in-the-loop safeguards.

Testing covered individual components, component integration, complete system workflows, usability, operational behaviour, acceptance criteria, security, reliability, and deployment. The combination of automated Pytest, Vitest, Playwright and optional Hypothesis testing with manual and requirement-level validation provided coverage across both technical correctness and user-facing behaviour.

The final Testing Event 2 report records all documented requirements and scope constraints as PASS. Based on these results, the system was considered ready for final project delivery, subject to the documented scope limitations and deployment configuration.

Automated results source: GitHub Actions CI on develop (6 August 2026) — backend-test, frontend-test, e2e-test jobs; Hypothesis fuzz suite run locally via make test-fuzz (8 August 2026).